

FlashAttention-2: 更快的注意力：更好的并行性与工作划分

Tri Dao^{1,2}

¹Department of Computer Science, Princeton University

²Department of Computer Science, Stanford University

trid@cs.stanford.edu

February 19, 2026

Abstract

在过去几年里，将 Transformer 扩展到更长的序列长度一直是一个关键问题，这有望提升语言建模与高分辨率图像理解性能，并解锁代码、音频与视频生成等新应用。注意力层是扩展到长序列时的主要瓶颈，因为其运行时间和内存开销都随序列长度呈二次增长。FlashAttention [5] 利用了 GPU 非对称的内存层次结构，在不做近似的前提下，实现了显著的内存节省（由二次降为线性）和运行加速（相对优化基线提升 2-4 $\times$ ）。然而，FlashAttention 相比优化后的矩阵乘（GEMM）仍然不够快，只能达到理论最大 FLOPs/s 的 25-40%。我们观察到，这种低效率来自 GPU 上不同 thread block 与 warp 之间次优的工作划分，导致要么占用率低，要么发生不必要的共享内存读写。我们提出 FlashAttention-2，通过更优的工作划分来解决这些问题。具体而言，我们 (1) 调整算法以减少非矩阵乘 FLOPs 数量；(2) 即使对单个头，也在不同 thread block 之间并行注意力计算以提高占用率；(3) 在每个 thread block 内，在不同 warp 之间分配工作以减少通过共享内存的通信。这些改进相较 FlashAttention 带来约 2 $\times$ 加速，在 A100 上达到理论最大 FLOPs/s 的 50-73%，并接近 GEMM 操作的效率。我们进一步在端到端 GPT 风格模型训练中实证验证：FlashAttention-2 在每张 A100 GPU 上可达到最高 225 TFLOPs/s 的训练速度（模型 FLOPs 利用率 72%）。¹

1 引言

提升 Transformer [18] 的上下文长度是一项挑战，因为其核心注意力层在运行时间和内存需求上都随输入序列长度二次增长。理想情况下，我们希望突破标准的 2k 序列长度限制，以训练能够理解长篇图书、高分辨率图像和长视频的模型。仅在过去一年中，就出现了多个上下文长度显著增加的语言模型：GPT-4 [12] 的上下文长度为 32k，MosaicML 的 MPT 为 65k，Anthropic 的 Claude 为 100k。长文档查询、故事写作等新兴应用也表明，对此类长上下文模型存在明确需求。

为了降低长上下文下注意力计算的开销，已有大量方法被提出用于近似注意力 [2, 3, 4, 8, 9, 14, 19, 20]。尽管这些方法在一些场景中得到应用，但据我们所知，大规模训练仍主要使用标准注意力。受此启发，Dao et al. [5] 提出通过重排注意力计算，并结合经典技术（分块、重计算）来显著提速，同时将内存开销从随序列长度二次增长降到线性增长。这在不做近似的前提下，相对优化基线带来 2-4 $\times$ 的端到端耗时加速和最高 10-20 $\times$ 的内存节省，因此 FlashAttention 已在 Transformer 的大规模训练和推理中得到广泛采用。

然而，随着上下文长度继续增加，FlashAttention 相比矩阵乘（GEMM）等其他基础算子仍不够高效。具体来说，尽管 FlashAttention 相比标准注意力实现已快 2-4 $\times$ ，其前向传播仍只能达到设备理论最大 FLOPs/s 的 30-50% (Fig. 5)，而反向传播更具挑战，在 A100 GPU 上仅达到最大吞吐的 25-35% (Fig. 6)。相比之下，优化后的 GEMM 可达到理论最大设备吞吐的 80-90%。通过细致剖析，我们发现 FlashAttention 在 GPU 不同 thread block 与 warp 之间的工作划分仍然次优，导致要么占用率不足，要么出现不必要的共享内存读写。

基于 FlashAttention，我们提出 FlashAttention-2，通过更好的并行策略和工作划分来解决这些挑战。

1. 在 Section 3.1 中，我们调整算法，在不改变输出的前提下降低非矩阵乘 FLOPs 数量。尽管非矩阵乘 FLOPs 仅占总 FLOPs 的一小部分，但由于 GPU 具有矩阵乘专用单元，这部分操作执行更慢；矩阵乘吞吐可比非矩阵乘高至 16 $\times$ 。因此，减少非矩阵乘 FLOPs、尽量把时间用于矩阵乘 FLOPs 非常重要。

¹FlashAttention-2 代码见 <https://github.com/Dao-AILab/flash-attention>

2. 我们提出在 batch 维度和头数维度之外，沿序列长度维度同时并行前向与反向传播。当序列较长（因此 batch size 往往较小）时，这会提升占用率（GPU 资源利用率）。
3. 即使在单个注意力计算块内部，我们也在 thread block 的不同 warp 之间划分工作，以减少通信和共享内存读写。

在 Section 4 中，我们实证验证 FlashAttention-2 相较 FlashAttention 也能带来显著加速。在不同设置（是否使用因果掩码、不同 head dimension）下的基准结果表明，FlashAttention-2 相比 FlashAttention 约有 2× 加速，前向传播最高达到理论最大吞吐的 73%，反向传播最高达到 63%。当用于端到端 GPT 风格模型训练时，我们在每张 A100 GPU 上达到最高 225 TFLOPs/s 的训练速度。

2 背景

我们先介绍 GPU 的性能特征与执行模型。随后描述标准注意力实现，以及 FlashAttention。

2.1 硬件特征

GPU 性能特征。 GPU 由计算单元（如浮点运算单元）和内存层次结构组成。大多数现代 GPU 都包含用于加速低精度矩阵乘的专用单元（如 Nvidia GPU 上用于 FP16/BF16 矩阵乘的 Tensor Core）。其内存层次主要包括高带宽内存（HBM）和片上 SRAM（即共享内存）。以 A100 为例：其 HBM 容量为 40-80GB，带宽为 1.5-2.0TB/s；同时有 108 个流式多处理器（SM），每个 SM 配有 192KB 片上 SRAM，估计带宽约 19TB/s [6, 7]。由于程序员无法直接控制 L2 缓存，在本文讨论中我们重点关注 HBM 与 SRAM。

执行模型。 GPU 使用大量线程来执行一个操作（称为 kernel）。线程被组织为 thread block，并调度到流式多处理器（SM）上运行。在每个 thread block 内，线程又分组为 warp（每组 32 个线程）。同一 warp 内线程可通过快速 shuffle 指令通信，或协同完成矩阵乘。同一 thread block 中不同 warp 可通过读写共享内存进行通信。每个 kernel 从 HBM 将输入加载到寄存器和 SRAM，完成计算后再把输出写回 HBM。

2.2 标准注意力实现

给定输入序列 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ ，其中 N 是序列长度、 d 是头维度，我们要计算注意力输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$ ：

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

其中 softmax 按行应用。² 对于多头注意力（MHA），同样的计算会在多个头上并行执行，并在 batch 维度（输入序列数）上并行。

注意力的反向传播如下。设 $d\mathbf{O} \in \mathbb{R}^{N \times d}$ 为某个损失函数对 $\mathbf{O}$ 的梯度，则由链式法则（即反向传播）有：

$$\begin{aligned} d\mathbf{V} &= \mathbf{P}^\top d\mathbf{O} \in \mathbb{R}^{N \times d} \\ d\mathbf{P} &= d\mathbf{O}\mathbf{V}^\top \in \mathbb{R}^{N \times N} \\ d\mathbf{S} &= d\text{softmax}(d\mathbf{P}) \in \mathbb{R}^{N \times N} \\ d\mathbf{Q} &= d\mathbf{S}\mathbf{K} \in \mathbb{R}^{N \times d} \\ d\mathbf{K} &= \mathbf{Q}d\mathbf{S}^\top \in \mathbb{R}^{N \times d}, \end{aligned}$$

其中 dsoftmax 是按行 softmax 的梯度（反向传播）。可推得：若对某向量 s 有 $p = \text{softmax}(s)$ ，在输出梯度为 dp 时，输入梯度为 $ds = (\text{diag}(p) - pp^\top)dp$ 。

标准注意力实现会将矩阵 $\mathbf{S}$ 与 $\mathbf{P}$ 物化到 HBM，这需要 $O(N^2)$ 内存。通常 $N \gg d$ （典型地 N 在 1k-8k 数量级， d 在 64-128 左右）。标准实现会先 (1) 调用矩阵乘（GEMM）子程序计算 $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$ 并写入 HBM；再 (2) 从 HBM 读取 $\mathbf{S}$ 计算 softmax 并将结果 $\mathbf{P}$ 写回 HBM；最后 (3) 调用 GEMM 得到 $\mathbf{O} = \mathbf{P}\mathbf{V}$ 。由于多数操作受内存带宽限制，大量内存访问会直接导致较慢的实际运行时间。此外，因为必须物化 $\mathbf{S}$ 和 $\mathbf{P}$ ，所需内存为 $O(N^2)$ 。并且为了在反向传播中计算梯度，还必须保存 $\mathbf{P} \in \mathbb{R}^{N \times N}$ 。

²为便于说明，我们省略了 $\mathbf{Q}\mathbf{K}^\top$ 的缩放（通常为 $1/d$ ），以及可选地施加在 $\mathbf{S}$ 上的逐元素掩码和/或施加在 $\mathbf{P}$ 上的 dropout。

2.3 FlashAttention

为了在 GPU 等硬件加速器上加速注意力, [5] 提出了在保持输出不变 (无近似) 的前提下减少内存读写的算法。

2.3.1 前向传播

FlashAttention 采用经典的分块 (tiling) 技术来减少内存 IO: (1) 将输入块从 HBM 读入 SRAM; (2) 对该块计算注意力; (3) 更新输出且不把大型中间矩阵 $\mathbf{S}$ 与 $\mathbf{P}$ 写回 HBM。由于 softmax 会耦合整行或行块, 在线 softmax [11, 13] 可将注意力计算拆成多个块, 并在末端对各块输出重标定, 从而得到正确结果 (无近似)。通过显著减少内存读写, FlashAttention 相对优化后的基线注意力实现可获得 2-4 $\times$ 的端到端加速。

我们介绍在线 softmax 技术 [11] 及其在注意力中的用法 [13]。为简化起见, 仅考虑注意力矩阵 $\mathbf{S}$ 的一个行块, 形如 $[\mathbf{S}^{(1)} \quad \mathbf{S}^{(2)}]$, 其中 $\mathbf{S}^{(1)}, \mathbf{S}^{(2)} \in \mathbb{R}^{B_r \times B_c}$, B_r, B_c 分别为行块和列块大小。我们希望计算这个行块的 softmax, 并与 value 相乘, value 形如 $\begin{bmatrix} \mathbf{V}^{(1)} \\ \mathbf{V}^{(2)} \end{bmatrix}$, 其中 $\mathbf{V}^{(1)}, \mathbf{V}^{(2)} \in \mathbb{R}^{B_c \times d}$ 。标准 softmax 计算如下:

$$\begin{aligned} m &= \max(\text{rowmax}(\mathbf{S}^{(1)}), \text{rowmax}(\mathbf{S}^{(2)})) \in \mathbb{R}^{B_r} \\ \ell &= \text{rowsum}(e^{\mathbf{S}^{(1)}-m}) + \text{rowsum}(e^{\mathbf{S}^{(2)}-m}) \in \mathbb{R}^{B_r} \\ \mathbf{P} &= [\mathbf{P}^{(1)} \quad \mathbf{P}^{(2)}] = \text{diag}(\ell)^{-1} \begin{bmatrix} e^{\mathbf{S}^{(1)}-m} & e^{\mathbf{S}^{(2)}-m} \end{bmatrix} \in \mathbb{R}^{B_r \times 2B_c} \\ \mathbf{O} &= [\mathbf{P}^{(1)} \quad \mathbf{P}^{(2)}] \begin{bmatrix} \mathbf{V}^{(1)} \\ \mathbf{V}^{(2)} \end{bmatrix} = \text{diag}(\ell)^{-1} e^{\mathbf{S}^{(1)}-m} \mathbf{V}^{(1)} + e^{\mathbf{S}^{(2)}-m} \mathbf{V}^{(2)} \in \mathbb{R}^{B_r \times d}. \end{aligned}$$

在线 softmax 则先按块计算 “局部” softmax, 再在末端重标定得到正确输出:

$$\begin{aligned} m^{(1)} &= \text{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r} \\ \ell^{(1)} &= \text{rowsum}(e^{\mathbf{S}^{(1)}-m^{(1)}}) \in \mathbb{R}^{B_r} \\ \tilde{\mathbf{P}}^{(1)} &= \text{diag}(\ell^{(1)})^{-1} e^{\mathbf{S}^{(1)}-m^{(1)}} \in \mathbb{R}^{B_r \times B_c} \\ \mathbf{O}^{(1)} &= \tilde{\mathbf{P}}^{(1)} \mathbf{V}^{(1)} = \text{diag}(\ell^{(1)})^{-1} e^{\mathbf{S}^{(1)}-m^{(1)}} \mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d} \\ m^{(2)} &= \max(m^{(1)}, \text{rowmax}(\mathbf{S}^{(2)})) = m \\ \ell^{(2)} &= e^{m^{(1)}-m^{(2)}} \ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)}-m^{(2)}}) = \text{rowsum}(e^{\mathbf{S}^{(1)}-m}) + \text{rowsum}(e^{\mathbf{S}^{(2)}-m}) = \ell \\ \tilde{\mathbf{P}}^{(2)} &= \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)}-m^{(2)}} \\ \mathbf{O}^{(2)} &= \text{diag}(\ell^{(1)}/\ell^{(2)})^{-1} \mathbf{O}^{(1)} + \tilde{\mathbf{P}}^{(2)} \mathbf{V}^{(2)} = \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(1)}-m} \mathbf{V}^{(1)} + \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)}-m} \mathbf{V}^{(2)} = \mathbf{O}. \end{aligned}$$

我们展示 FlashAttention 如何使用在线 softmax 来实现分块 (Fig. 1), 从而减少内存读写。

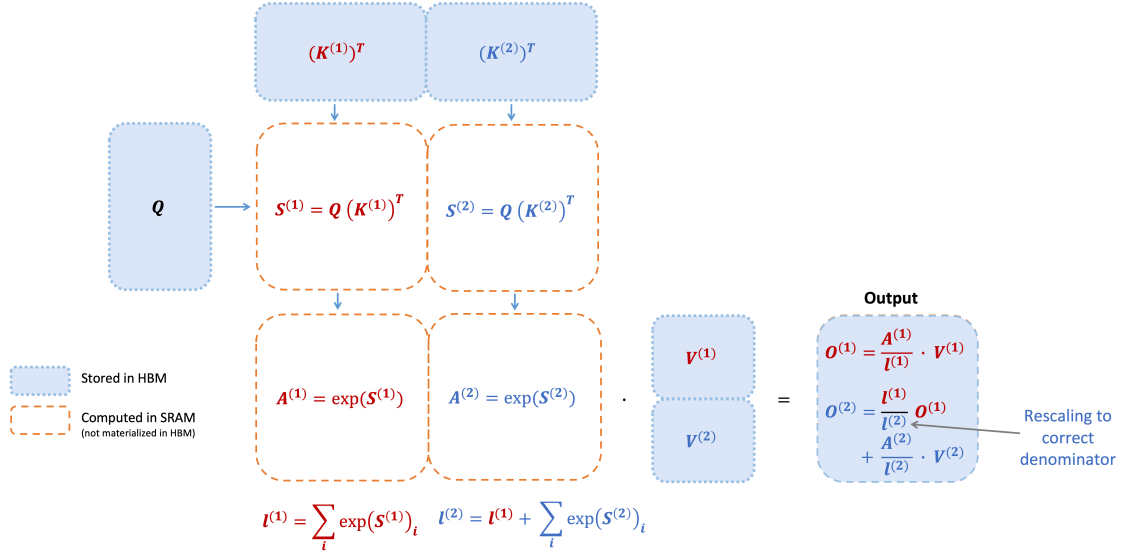
2.3.2 反向传播

在反向传播中, 一旦输入块 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 已加载到 SRAM, FlashAttention 会重计算注意力矩阵 $\mathbf{S}$ 与 $\mathbf{P}$, 从而避免存储大型中间值。由于不必保存大小为 $N \times N$ 的大矩阵 $\mathbf{S}$ 和 $\mathbf{P}$, FlashAttention 可根据序列长度获得 10-20 $\times$ 的内存节省 (所需内存随序列长度 N 线性增长, 而非二次增长)。反向传播也因减少内存读写而获得 2-4 $\times$ 的端到端加速。

反向传播将分块技术应用于 Section 2.2 的方程。尽管从概念上看反向传播比前向更简单 (没有 softmax 重标定), 但实现上要复杂得多。这是因为反向传播需要在 SRAM 中保留更多数值来执行 5 次矩阵乘, 而前向仅需 2 次。

3 FlashAttention-2 : 算法、并行性与工作划分

我们介绍 FlashAttention-2 算法, 其中包含了对 FlashAttention 的若干改动, 以减少非矩阵乘 FLOPs 的数量。随后我们说明如何在不同 thread block 上并行计算, 以充分利用 GPU 资源。最后我们说明如何在单个 thread block 内的不同 warp 之间划分工作, 以减少共享内存访问量。这些改进带来了 2-3 $\times$ 的速度提升, Section 4 给出了验证结果。



3.1 算法

我们对 FlashAttention 的算法进行了调整，以减少非矩阵乘 FLOPs 的数量。原因在于现代 GPU 拥有专用计算单元（例如 Nvidia GPU 的 Tensor Cores），使矩阵乘显著更快。以 A100 为例，其 FP16/BF16 矩阵乘理论峰值吞吐为 312 TFLOPs/s，而非矩阵乘 FP32 仅为 19.5 TFLOPs/s。换个角度看，每个非矩阵乘 FLOP 的代价约为矩阵乘 FLOP 的 16 $\times$ 。为了保持高吞吐（例如超过理论峰值 TFLOPs/s 的 50%），我们希望尽可能把时间花在矩阵乘 FLOPs 上。

3.1.1 前向传播

我们回顾 Section 2.3 中的在线 softmax 技巧，并做了两点小改动来减少非矩阵乘 FLOPs：

1. 输出更新中不必将两项都乘以 $\text{diag}(\ell^{(2)})^{-1}$ ：

$$\mathbf{O}^{(2)} = \text{diag}(\ell^{(1)}/\ell^{(2)})^{-1} \mathbf{O}^{(1)} + \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)} - m^{(2)}} \mathbf{V}^{(2)}.$$

我们可以改为维护一个“未缩放”的 $\mathbf{O}^{(2)}$ ，并保留统计量 $\ell^{(2)}$ ：

$$\tilde{\mathbf{O}}^{(2)} = \text{diag}(\ell^{(1)})^{-1} \mathbf{O}^{(1)} + e^{\mathbf{S}^{(2)} - m^{(2)}} \mathbf{V}^{(2)}.$$

仅在循环最后一步，再把最终的 $\tilde{\mathbf{O}}^{(\text{last})}$ 乘以 $\text{diag}(\ell^{(\text{last})})^{-1}$ 得到正确输出。

2. 在反向传播中无需同时保存最大值 $m^{(j)}$ 与指数和 $\ell^{(j)}$ 。我们只需存储 $\text{logsumexp}: L^{(j)} = m^{(j)} + \log(\ell^{(j)})$ 。

在 Section 2.3 的 2 个块简化情形中，在线 softmax 现在可写为：

$$\begin{aligned}
m^{(1)} &= \text{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r} \\
\ell^{(1)} &= \text{rowsum}(e^{\mathbf{S}^{(1)} - m^{(1)}}) \in \mathbb{R}^{B_r} \\
\mathbf{O}^{(1)} &= e^{\mathbf{S}^{(1)} - m^{(1)}} \mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d} \\
m^{(2)} &= \max(m^{(1)}, \text{rowmax}(\mathbf{S}^{(2)})) = m \\
\ell^{(2)} &= e^{m^{(1)} - m^{(2)}} \ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)} - m^{(2)}}) = \text{rowsum}(e^{\mathbf{S}^{(1)} - m} + e^{\mathbf{S}^{(2)} - m}) = \ell \\
\tilde{\mathbf{P}}^{(2)} &= \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)} - m^{(2)}} \\
\tilde{\mathbf{O}}^{(2)} &= \text{diag}(e^{m^{(1)} - m^{(2)}})^{-1} \tilde{\mathbf{O}}^{(1)} + e^{\mathbf{S}^{(2)} - m^{(2)}} \mathbf{V}^{(2)} = e^{s^{(1)} - m} \mathbf{V}^{(1)} + e^{s^{(2)} - m} \mathbf{V}^{(2)} \\
\mathbf{O}^{(2)} &= \text{diag}(\ell^{(2)})^{-1} \tilde{\mathbf{O}}^{(2)} = \mathbf{O}.
\end{aligned}$$

我们在 Algorithm 1 中给出完整的 FlashAttention-2 前向传播。

Algorithm 1 FlashAttention-2 前向传播

Require: HBM 中矩阵 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ ，块大小 B_c, B_r 。

- 1: 将 $\mathbf{Q}$ 划分为 $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ 个块 $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ ，每块大小为 $B_r \times d$ ；并将 $\mathbf{K}, \mathbf{V}$ 划分为 $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ 个块 $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ 与 $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$ ，每块大小为 $B_c \times d$ 。
 - 2: 将输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$ 划分为 T_r 个块 $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ ，每块大小为 $B_r \times d$ ；并将 logsumexp 向量 L 划分为 T_r 个块 $L_1, \dots, L_{T_r}$ ，每块大小为 B_r 。
 - 3: **for** $1 \leq i \leq T_r$ **do**
 - 4: 将 $\mathbf{Q}_i$ 从 HBM 读入片上 SRAM。
 - 5: 在片上初始化 $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}$, $\ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}$, $m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$ 。
 - 6: **for** $1 \leq j \leq T_c$ **do**
 - 7: 将 $\mathbf{K}_j, \mathbf{V}_j$ 从 HBM 读入片上 SRAM。
 - 8: 在片上计算 $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$ 。
 - 9: 在片上计算 $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$ (逐元素), $\ell_i^{(j)} = e^{m_i^{(j-1)} - m_i^{(j)}} \ell_i^{(j-1)} + \text{row sum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$ 。
 - 10: 在片上计算 $\mathbf{O}_i^{(j)} = \text{diag}(e^{m_i^{(j-1)} - m_i^{(j)}})^{-1} \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$ 。
 - 11: **end for**
 - 12: 在片上计算 $\mathbf{O}_i = \text{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$ 。
 - 13: 在片上计算 $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$ 。
 - 14: 将 $\mathbf{O}_i$ 写回 HBM，作为 $\mathbf{O}$ 的第 i 个块。
 - 15: 将 L_i 写回 HBM，作为 L 的第 i 个块。
 - 16: **end for**
 - 17: 返回输出 $\mathbf{O}$ 与 logsumexp 向量 L 。
-

因果掩码。 注意力的一个常见应用是自回归语言建模，此时需要对注意力矩阵 $\mathbf{S}$ 施加因果掩码（即任意 $j > i$ 的元素 $\mathbf{S}_{ij}$ 被设为 $-\infty$ ）。

1. 由于 FlashAttention 与 FlashAttention-2 本身按块计算，对于所有“列索引整体大于行索引”的块（当序列很长时大约占一半），我们可以直接跳过该块计算。与不使用因果掩码的注意力相比，这可带来约 1.7-1.8× 加速。
2. 对于行索引严格小于列索引的块，不需要再显式应用因果掩码。这意味着对每一行而言，我们只需在 1 个块上应用因果掩码（假设方形块）。

正确性、运行时间与内存需求。 与 FlashAttention 一样，Algorithm 1 在无近似条件下返回正确输出 $\mathbf{O} = \text{softmax}(\mathbf{Q}\mathbf{K}^T)\mathbf{V}$ ，使用 $O(N^2d)$ FLOPs，且除输入输出外仅需 $O(N)$ 额外内存（用于存储 logsumexp L ）。其证明与 Dao et al. [5, Theorem 1] 的证明几乎相同，这里省略。

3.1.2 反向传播

FlashAttention-2 的反向传播与 FlashAttention 基本一致。我们仅做了一个小改动：在 softmax 中仅使用按行 logsumexp L ，而不再同时使用按行最大值与按行指数和。为完整起见，我们在 Algorithm 2 中给出反向传播描述。

Algorithm 2 FlashAttention-2 反向传播

Require: HBM 中矩阵 $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ ，HBM 中向量 $L \in \mathbb{R}^N$ ，块大小 B_c, B_r 。

- 1: 将 $\mathbf{Q}$ 划分为 $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ 个块 $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ ，每块大小为 $B_r \times d$ ；并将 $\mathbf{K}, \mathbf{V}$ 划分为 $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ 个块 $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ 与 $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$ ，每块大小为 $B_c \times d$ 。
 - 2: 将 $\mathbf{O}$ 划分为 T_r 个块 $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ ，每块大小为 $B_r \times d$ ；将 $\mathbf{dO}$ 划分为 T_r 个块 $\mathbf{dO}_1, \dots, \mathbf{dO}_{T_r}$ ，每块大小为 $B_r \times d$ ；并将 L 划分为 T_r 个块 $L_1, \dots, L_{T_r}$ ，每块大小为 B_r 。
 - 3: 在 HBM 中初始化 $\mathbf{dQ} = (0)_{N \times d}$ ，并将其划分为 T_r 个块 $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ ，每块大小为 $B_r \times d$ 。将 $\mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$ 划分为 T_c 个块 $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ 与 $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$ ，每块大小为 $B_c \times d$ 。
 - 4: 计算 $D = \text{rowsum}(\mathbf{dO} \circ \mathbf{O}) \in \mathbb{R}^d$ （逐元素乘），将 D 写回 HBM，并划分为 T_r 个块 $D_1, \dots, D_{T_r}$ ，每块大小为 B_r 。
 - 5: **for** $1 \leq j \leq T_c$ **do**
 - 6: 将 $\mathbf{K}_j, \mathbf{V}_j$ 从 HBM 读入片上 SRAM。
 - 7: 在 SRAM 中初始化 $\mathbf{dK}_j = (0)_{B_c \times d}, \mathbf{dV}_j = (0)_{B_c \times d}$ 。
 - 8: **for** $1 \leq i \leq T_r$ **do**
 - 9: 将 $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, L_i, D_i$ 从 HBM 读入片上 SRAM。
 - 10: 在片上计算 $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$ 。
 - 11: 在片上计算 $\mathbf{P}_i^{(j)} = \exp(\mathbf{S}_{ij} - L_i) \in \mathbb{R}^{B_r \times B_c}$ 。
 - 12: 在片上计算 $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_i^{(j)})^T \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$ 。
 - 13: 在片上计算 $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i \mathbf{V}_j^T \in \mathbb{R}^{B_r \times B_c}$ 。
 - 14: 在片上计算 $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ (\mathbf{dP}_i^{(j)} - D_i) \in \mathbb{R}^{B_r \times B_c}$ 。
 - 15: 将 $\mathbf{dQ}_i$ 从 HBM 读入 SRAM，然后在片上更新 $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}_i^{(j)} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$ ，并写回 HBM。
 - 16: 在片上计算 $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \mathbf{dS}_i^{(j)T} \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$ 。
 - 17: **end for**
 - 18: 将 $\mathbf{dK}_j, \mathbf{dV}_j$ 写回 HBM。
 - 19: **end for**
 - 20: 返回 $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$ 。
-

多查询注意力与分组查询注意力。 多查询注意力 (MQA) [15] 与分组查询注意力 (GQA) [1] 是注意力的两种变体：多个 query 头共享同一个 key/value 头，以在推理时减小 KV cache 的大小。在计算中，我们无需显式复制 key/value 头，而是通过隐式操控头索引来实现等价计算。在反向传播中，需要对被隐式复制的不同头对应的梯度 $\mathbf{dK}$ 和 $\mathbf{dV}$ 进行求和。

3.2 并行性

FlashAttention 的第一版仅在 batch 大小与头数维度上并行。我们使用 1 个 thread block 处理 1 个注意力头，因此总 thread block 数为 batch size · number of heads。每个 thread block 会被调度到一个 SM 上执行；例如 A100 上共有 108 个 SM。当该数量较大（例如 ≥ 80 ）时，这种调度效率较高，因为几乎能用满 GPU 的计算资源。

在长序列场景下（通常意味着 batch size 或头数较小），为了更好利用 GPU 的多处理器，我们进一步沿序列长度维度并行。这在该场景下会带来显著加速。

前向传播。 我们观察到外层循环（沿序列长度）是天然可并行的，可将其调度到彼此无需通信的不同 thread block 上。与 FlashAttention 一样，我们也在 batch 维度和头数维度并行。序列长度维度上的额外并行，在 batch size 和头数较小时可提升占用率（GPU 资源使用比例），从而带来加速。

这些思想包括：交换循环顺序（外层遍历行块、内层遍历列块，而不是 FlashAttention 原论文中的相反顺序），以及沿序列长度维度并行。这些最早由 Phil Tillet 在 Triton [17] 实现中提出并实现。³

反向传播。 注意不同列块之间唯一共享的计算是在 Algorithm 2 中更新 dQ ：需要将 dQ_i 从 HBM 读入 SRAM，然后在片上更新 $dQ_i \leftarrow dQ_i + dS_i^{(j)} K_j$ ，再写回 HBM。因此我们同样沿序列长度维度并行，并在反向传播中为每个列块调度 1 个 thread block。我们使用原子加（atomic add）让不同 thread block 之间协同更新 dQ 。

我们在 Fig. 2 中展示并行化方案。

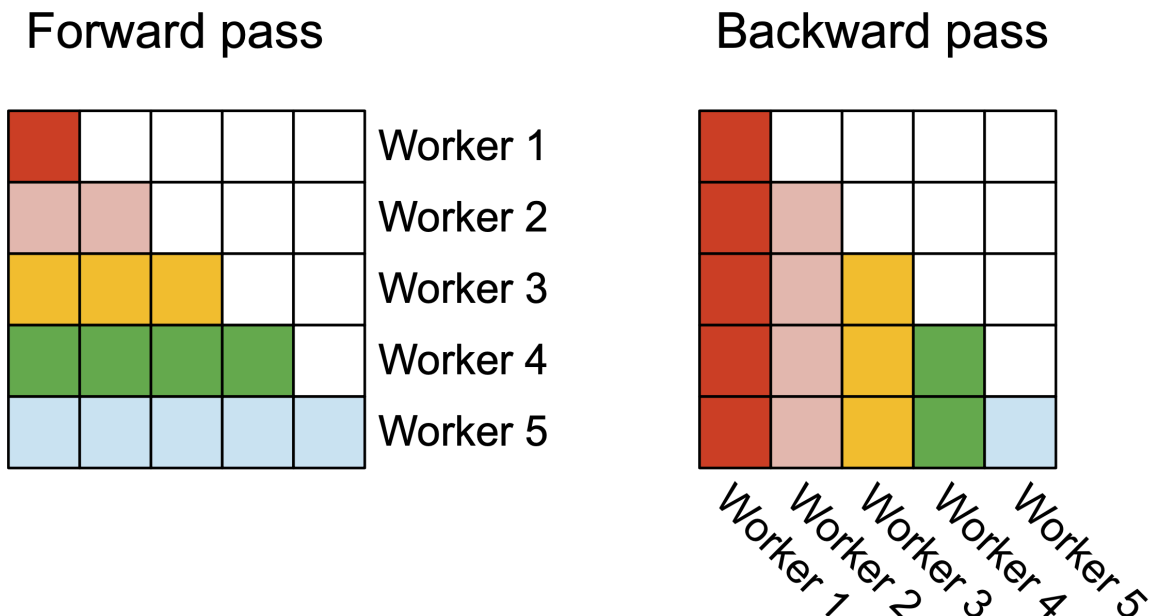


Figure 2: 前向传播（左）中，我们对 worker（thread block）并行化，每个 worker 负责注意力矩阵的一行。反向传播（右）中，每个 worker 负责注意力矩阵的一列。

3.3 Warp 间工作划分

Section 3.2 介绍了 thread block 级别的调度；即使在每个 thread block 内，我们仍需决定如何在不同 warp 之间划分工作。我们通常在每个 thread block 中使用 4 或 8 个 warp，其划分方式见 Fig. 3。

前向传播。 对每个块，FlashAttention 会将 K 和 V 分给 4 个 warp，而 Q 对所有 warp 可见。每个 warp 先做乘法得到 QK^T 的一个切片，再与 V 的一个切片相乘，并通过通信把结果累加。这被称为“split-K”方案。但该方案效率不高，因为所有 warp 都需要把中间结果写到共享内存、同步，再把中间结果相加。这些共享内存读写拖慢了 FlashAttention 的前向传播。

在 FlashAttention-2 中，我们改为将 Q 在 4 个 warp 之间划分，而 K 与 V 对所有 warp 可见。每个 warp 在得到 QK^T 的一个切片后，只需与共享的 V 切片相乘，即可得到对应的输出切片。不再需要 warp 间通信。共享内存读写减少后可带来加速（Section 4）。

反向传播。 在反向传播中同样地，我们也选择避免“split-K”的 warp 划分方式。不过由于各输入与梯度之间依赖关系更复杂 $Q, K, V, O, dO, dQ, dK, dV$ ，仍需要一定同步。尽管如此，避免“split-K”仍可减少共享内存读写，并再次带来加速（Section 4）。

³<https://github.com/openai/triton/blob/main/python/tutorials/06-fused-attention.py>

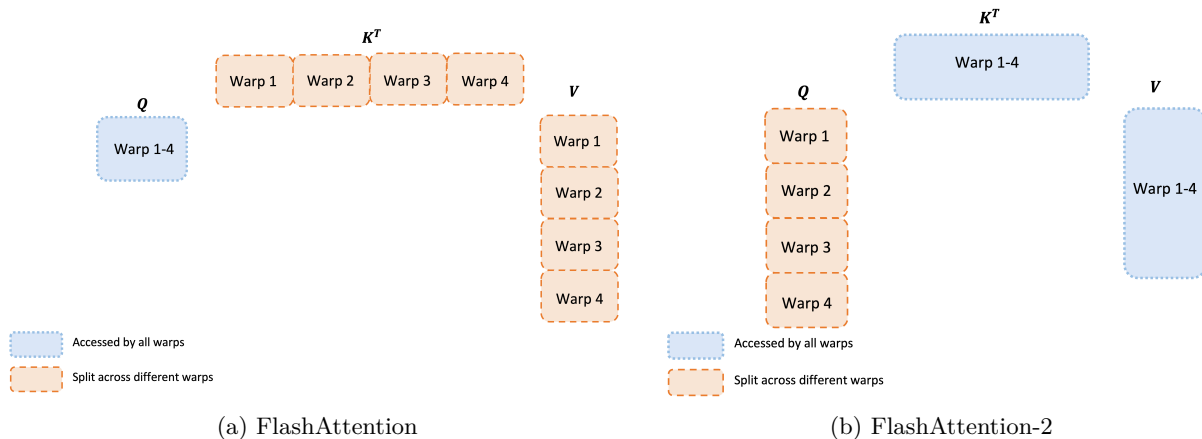


Figure 3: 前向传播中不同 warp 之间的工作划分

块大小调优 增大块大小通常会减少共享内存加载/存储次数，但会增加所需寄存器数量和共享内存总量。当块大小超过某个阈值后，寄存器溢出（register spilling）会导致明显降速，或者所需共享内存超过 GPU 可用容量，使 kernel 根本无法运行。我们通常选择大小为 $\{64, 128\} \times \{64, 128\}$ 的块，具体取决于头维度 d 与设备共享内存大小。

由于块大小本质上只有 4 种选择，我们针对每个头维度手工调优；但这类工作可受益于自动调优，以减少人工成本。这留作未来工作。

4 实证验证

我们评估使用 FlashAttention-2 训练 Transformer 模型的效果。

- **注意力基准测试。** 我们测量 FlashAttention-2 在不同序列长度下的运行时间，并与 PyTorch 标准实现、FlashAttention 以及 Triton 中的 FlashAttention 进行比较。结果确认：FlashAttention-2 相比 FlashAttention 快 1.7-3.0 $\times$ ，相比 Triton 中的 FlashAttention 快 1.3-2.5 $\times$ ，相比标准注意力实现快 3-10 $\times$ 。在 A100 GPU 上，FlashAttention-2 最高可达 230 TFLOPs/s，即理论最大 TFLOPs/s 的 73%。
- **端到端训练速度** 当用于端到端训练 1.3B 与 2.7B 参数规模、序列长度为 2k 或 8k 的 GPT 风格模型时，FlashAttention-2 相比 FlashAttention 最高加速 1.3 $\times$ ，相比不使用 FlashAttention 的基线最高加速 2.8 $\times$ 。FlashAttention-2 在每张 A100 GPU 上最高达到 225 TFLOPs/s（模型 FLOPs 利用率 72%）。

4.1 注意力基准测试

我们在 A100 80GB SXM4 GPU 上，在不同设置（不使用/使用因果掩码、head dimension 为 64 或 128）下测量不同注意力方法的运行时间。结果见 Fig. 4、Fig. 5 与 Fig. 6。结果显示，FlashAttention-2 相比 FlashAttention 及 xformers 中的 FlashAttention（“cutlass”实现）约快 2 $\times$ 。在前向传播中，FlashAttention-2 相比 Triton 中的 FlashAttention 约快 1.3-1.5 $\times$ ；在反向传播中约快 2 $\times$ 。与 PyTorch 标准注意力实现相比，FlashAttention-2 最高可快 10 $\times$ 。

基准设置如下：序列长度从 512、1k、...、16k 变化，并设置 batch size 使总 token 数为 16k。隐藏维度设为 2048，head dimension 取 64 或 128（即分别为 32 个头或 16 个头）。前向传播 FLOPs 按下式计算：

$$4 \cdot \text{seqlen}^2 \cdot \text{head dimension} \cdot \text{number of heads}.$$

使用因果掩码时，该值再除以 2，以反映大约仅有一半元素会被计算。反向传播 FLOPs 则取前向 FLOPs 的 2.5 倍（由于前向有 2 次 matmul，反向因重计算有 5 次 matmul）。

仅将同一实现直接运行在 H100 GPU 上（未使用 TMA、第四代 Tensor Core 等新特性专用指令）时，我们可获得最高 335 TFLOPs/s (Fig. 7)。我们预计若使用新指令，还能在 H100 上再获得 1.5x-2x 的加速，这留作未来工作。

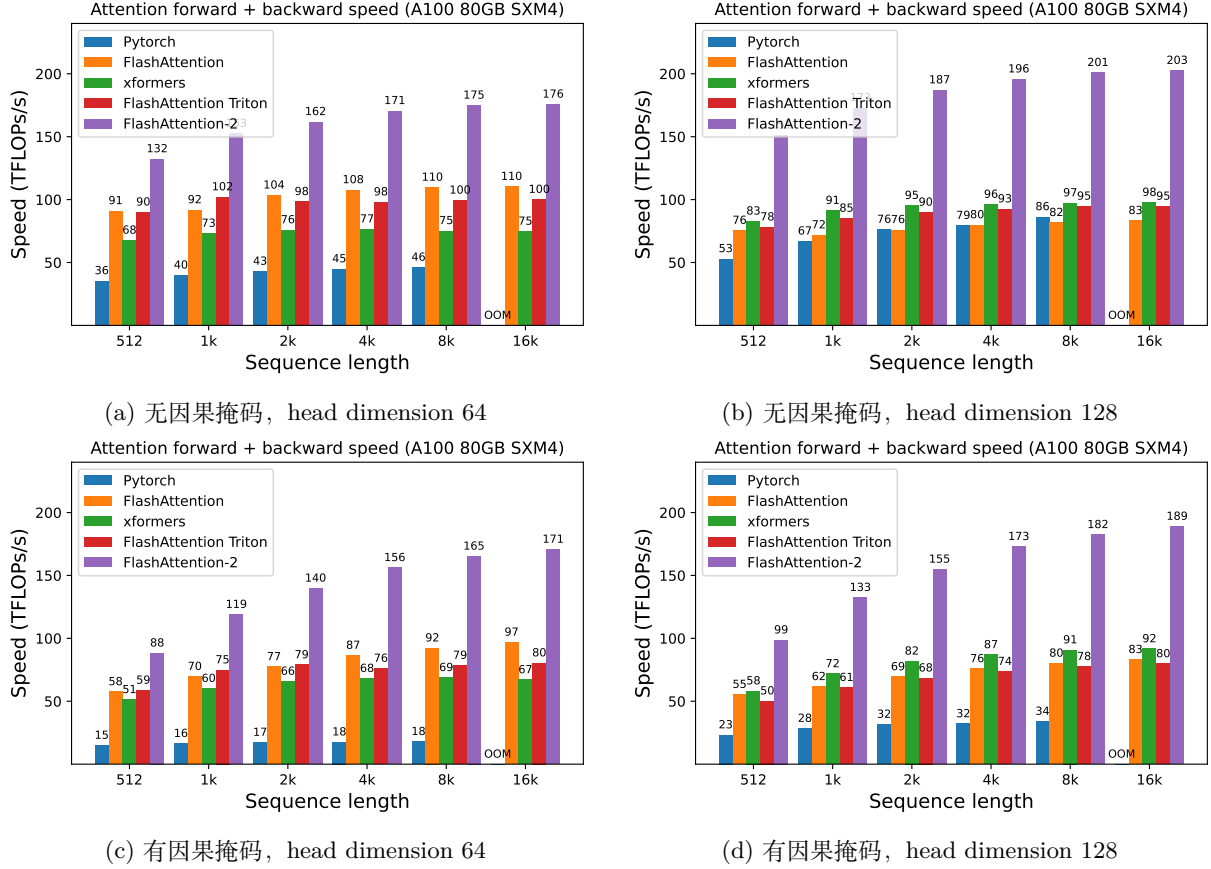


Figure 4: A100 GPU 上注意力前向 + 反向速度

4.2 端到端性能

我们在 8×A100 80GB SXM 上, 测量参数规模为 1.3B 或 2.7B 的 GPT 风格模型训练吞吐。如 Table 1 所示, FlashAttention-2 相比不使用 FlashAttention 的基线可达 2.8× 加速, 相比 FlashAttention 可达 1.3× 加速, 在每张 A100 GPU 上最高达到 225 TFLOPs/s。

需要说明的是, 我们按下式计算 FLOPs, 遵循 Megatron-LM [16] (以及许多其他论文与库):

$$6 \cdot \text{seqlen} \cdot \text{number of params} + 12 \cdot \text{number of layers} \cdot \text{hidden dim} \cdot \text{seqlen}^2.$$

第一项对应权重与输入相乘的 FLOPs, 第二项对应注意力 FLOPs。不过也可以认为第二项应减半, 因为在因果掩码下大约只需计算一半注意力元素。为保持一致性, 我们采用文献中的公式 (即将注意力 FLOPs 再除以 2)。

Table 1: 8×A100 GPU 上 GPT 风格模型训练速度 (TFLOPs/s/GPU)。FlashAttention-2 最高达到 225 TFLOPs/s (模型 FLOPs 利用率 72%)。对比基线为不使用 FlashAttention 的实现。

模型	不使用 FlashAttention	FlashAttention	FlashAttention-2
GPT3-1.3B 2k 上下文	142 TFLOPs/s	189 TFLOPs/s	196 TFLOPs/s
GPT3-1.3B 8k 上下文	72 TFLOPs/s	170 TFLOPs/s	220 TFLOPs/s
GPT3-2.7B 2k 上下文	149 TFLOPs/s	189 TFLOPs/s	205 TFLOPs/s
GPT3-2.7B 8k 上下文	80 TFLOPs/s	175 TFLOPs/s	225 TFLOPs/s

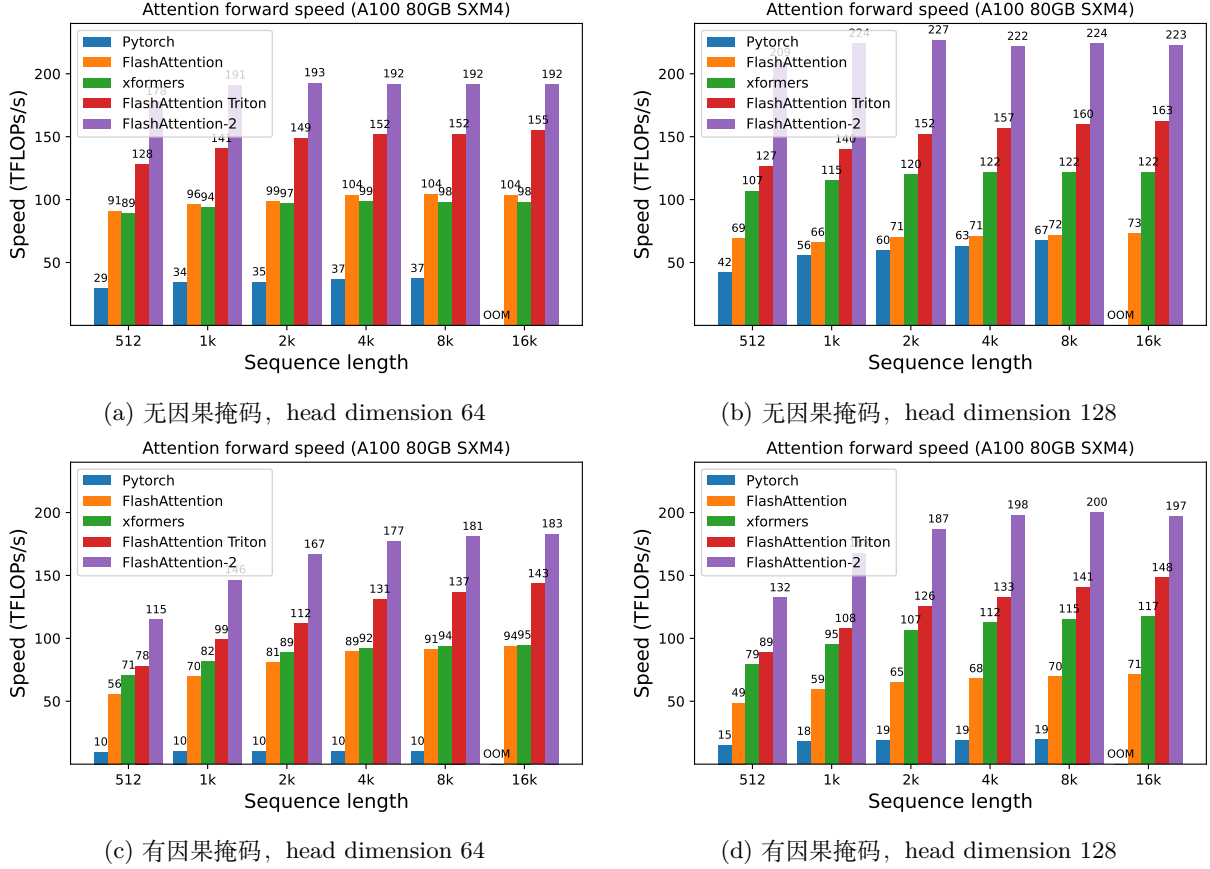


Figure 5: A100 GPU 上注意力前向速度

5 讨论与未来方向

FlashAttention-2 比 FlashAttention 快 2×，这意味着我们可以用与此前训练 8k 上下文模型相同的成本，训练 16k 上下文模型。我们期待这项能力被用于理解长篇图书和报告、高分辨率图像、音频与视频。FlashAttention-2 也将加速现有模型的训练、微调和推理。

在近期，我们计划与研究者和工程师合作，让 FlashAttention 可广泛应用于不同设备（例如 H100 GPU、AMD GPU）以及 FP8 等新数据类型。作为下一步，我们计划针对 H100 GPU 优化 FlashAttention-2，以利用新硬件特性（TMA、第四代 Tensor Core、FP8）。将 FlashAttention-2 的底层优化与更高层算法改进（例如局部注意力、扩张注意力、块稀疏注意力）结合，可能使我们能够训练上下文更长的 AI 模型。我们也期待与编译器研究者合作，让这些优化技术更易于编程实现。

致谢

我们感谢 Phil Tillet 和 Daniel Haziza，他们分别在 Triton [17] 和 xformers 库 [10] 中实现了 FlashAttention 的版本。FlashAttention-2 的提出也受到了不同注意力实现思路交流的启发。我们感谢 Nvidia CUTLASS 团队（尤其是 Vijay Thakkar、Cris Cecka、Haicheng Wu 和 Andrew Kerr）提供 CUTLASS 库，特别是 CUTLASS 3.x 版本；该版本为 FlashAttention-2 的实现提供了清晰抽象和强大构件。我们感谢 Driss Guessous 将 FlashAttention 集成到 PyTorch。FlashAttention-2 还受益于与 Phil Wang、Markus Rabe、James Bradbury、Young-Jun Ko、Julien Launay、Daniel Hesslow、Micha”el Benesty、Horace He、Ashish Vaswani 和 Erich Elsen 的讨论。感谢 Stanford CRFM 与 Stanford NLP 提供算力支持。我们感谢 Dan Fu 和 Christopher Ré 在这条“面向硬件高效算法设计”研究路线上的合作、建设性反馈与持续鼓励。我们也感谢 Albert Gu 与 Beidi Chen 对该技术报告早期草稿提出的宝贵建议。

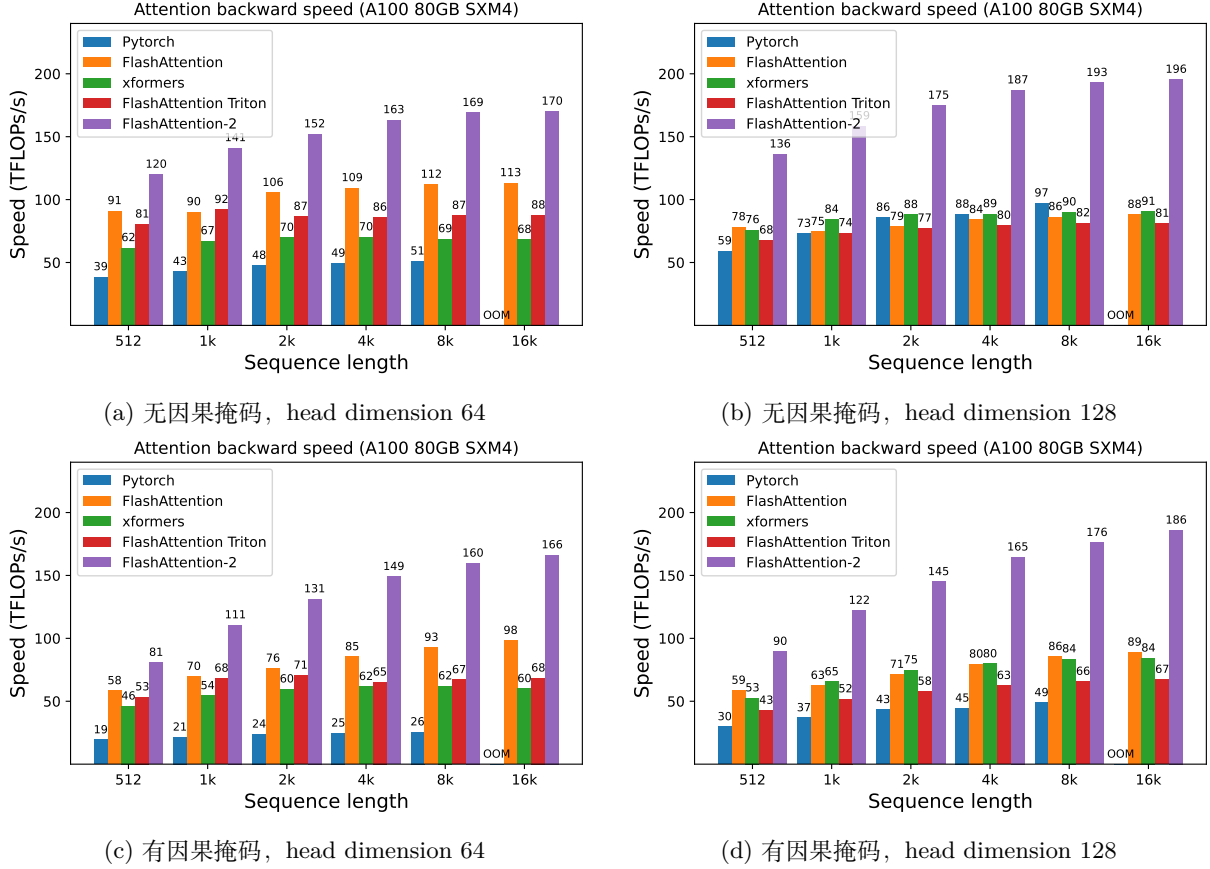


Figure 6: A100 GPU 上注意力反向速度

References

- [1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.
- [2] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [3] Beidi Chen, Tri Dao, Eric Winsor, Zhao Song, Atri Rudra, and Christopher Ré. Scatterbrain: Unifying sparse and low-rank attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [4] Krzysztof Marcin Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2020.
- [5] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.
- [6] Zhe Jia and Peter Van Sandt. Dissecting the Ampere GPU architecture via microbenchmarking. GPU Technology Conference, 2021.
- [7] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P Scarpazza. Dissecting the nvidia Volta GPU architecture via microbenchmarking. *arXiv preprint arXiv:1804.06826*, 2018.

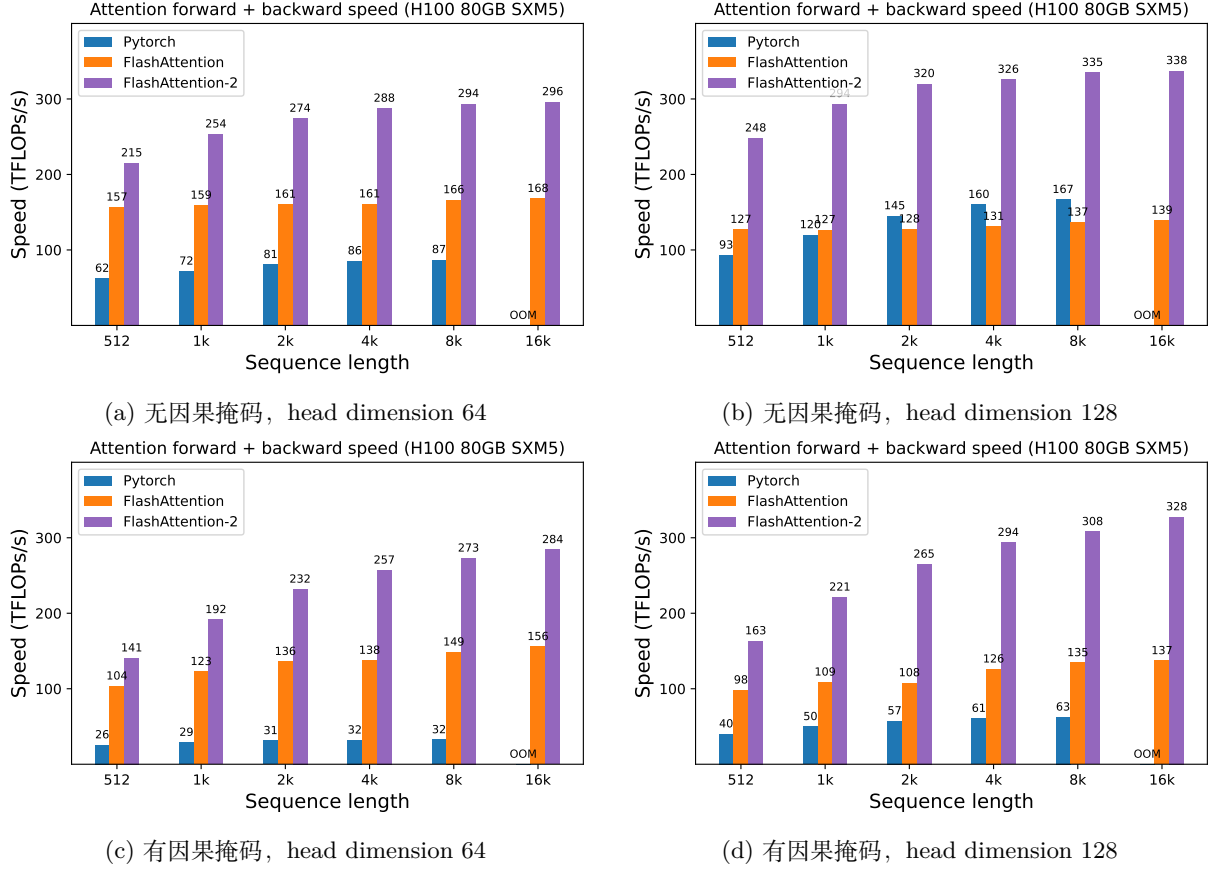


Figure 7: H100 GPU 上注意力前向 + 反向速度

- [8] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pages 5156–5165. PMLR, 2020.
- [9] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *The International Conference on Machine Learning (ICML)*, 2020.
- [10] Benjamin Lefaudeux, Francisco Massa, Diana Liskovich, Wenhan Xiong, Vittorio Caggiano, Sean Naren, Min Xu, Jieru Hu, Marta Tintore, Susan Zhang, Patrick Labatut, and Daniel Haziza. xformers: A modular and hackable transformer modelling library. <https://github.com/facebookresearch/xformers>, 2022.
- [11] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.
- [12] OpenAI. Gpt-4 technical report. *ArXiv*, abs/2303.08774, 2023.
- [13] Markus N Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.
- [14] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9: 53–68, 2021.
- [15] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

- [16] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [17] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.
- [18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [19] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.
- [20] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33, 2020.